Infrastructure as Code Using Terraform

INSTRUCTOR

KUSEH SIMON WEWOLIAMO

EMAIL

kusehsw@gmail.com

LINKEDIN

linkedin.com/in/kuseh-simon-wewoliamo

GITHUB

github.com/kodcapsule

Course Overview

- This course is designed to help practitioners to design, deploy, secure, automate and maintain cloud infrastructure using Terraform.
- Learners will move from manual provisioning of cloud infrastructure to writing their first terraform configuration, building production-grade modules, implementing IaC in CI/CD pipelines, AI-assisted risk review to Terraform plans.

Course Outline

WEEK / SESSION	HOURS	TOPICS	FORMART
Week 1 / S1	3 hrs	Introduction to IaC, Terraform Fundamentals & HCL	Lecture + Lab
Week 2 / S2	3 hrs	State Management & Environment Separation	Lecture + Lab
Week 3 / S3	3 hrs	Modules, Refactoring & Registry	Lecture + Lab
Week 4 / S4	3 hrs	CI/CD, Drift Detection & Provider Auth	Lecture + Lab
Week 5 / S5	3 hrs	Security, Cost Preview, AI Integration & Platforms	Lecture + Demo

Learning Objectives

- Write HCL configurations using providers, resources, data sources, variables, and outputs.
- Manage Terraform state safely — locally, remotely
- Author and publish reusable modules, following versioning and registry best practices.
- Execute plan, apply, and destroy workflows both locally and within CI/CD pipelines (GitHub Actions).
- Detect and reconcile infrastructure drift.
- Authenticate providers securely using IAM roles and OIDC
- Apply security scanning (tfsec, Checkov, OPA/Conftest) and policy enforcement (Sentinel) to Terraform code.
- Preview infrastructure costs before deployment using Infracost.
- Understand and evaluate Terraform Cloud and Spacelift for team-scale workflows.
- Use AI agents to read Terraform plans and flag risky changes before apply.

Recommended Course Reading Materials

1. HashiCorp Official Documentation
2. HashiCorp Learn: Terraform Getting Started on AWS (official tutorial series)
3. Terraform: Up & Running. Yevgeniy Brikman
4. AWS Well-Architected Framework — Infrastructure as Code section

Tools, Environment & Prerequisites

- **Prerequisites**
  1. Understanding of cloud computing concepts (AWS)
- **Tools, Environment**
  1. Terraform
  2. AWS CLI v2.
  3. Git & GITHUB Account
  4. A code editor: (VS Code)

Introduction to IaC, Terraform Fundamentals & HCL

WEEK 1 / S1

Week 1 / S1 Outline

- Introduction to Infrastructure as Code (IaC)
- HCL Syntax, Expressions, Functions
- Terraform Basics
- Lab 1A
- Lab 1B

Introduction to Infrastructure as Code (IaC)

What is infrastructure as code (IaC)?

- Infrastructure as Code (IaC), is an approach where you write and execute code to define, deploy, update, and destroy your infrastructure.
- IaC is a mindset where you treat all aspects of operations (servers, databases, networks) as software.
- Instead of manually provisioning resources through console interfaces aka "ClickOps" or ad-hoc scripts, IaC allows you to define infrastructure using code, bringing software development practices to infrastructure management

Introduction to Infrastructure as Code (IaC)

What are the benefits of using IaC

- **Consistency**: Eliminates configuration drift and ensures consistent environments
- **Repeatability**: Easily recreate environments with the same configurations
- **Scalability**: Efficiently scale infrastructure by replicating configurations
- **Version Control**: Track changes, rollback when needed, and understand the evolution of your infrastructure
- **Documentation**: The code itself serves as documentation for your infrastructure other benefits include, speed and safty, and easy documentation

Introduction to Infrastructure as Code (IaC)

Declarative vs Imperative Approaches

- **Declarative** ("what" you want): With declarative code you declare the end state that you want, and the tool will take all the necessary steps to get to the desired state.
- **Imperative (“**how” to create it): Describes a series of steps to follow to reach a goal.

Introduction to Infrastructure as Code (IaC)

IaC Tools

There are five categories of IaC tools:

1. **Configuration management tools (Chef, Puppet, Ansible):** As configuration management tools, Chef, Puppet, and Ansible are designed to install and maintain software on servers.
2. **Provisioning tools (Terraform, CloudFormation, Pulumi):** These tools are responsible for creating infrastructure resources(Servers, databases, caches, load balancers,)
3. **Ad hoc scripts (e.g., Bash, Ruby, Python):** Ad hoc scripts work well for short-term, one-time tasks, but if you plan to manage your entire infrastructure as code, it is best to utilize an IaC tool designed specifically for that purpose.
4. **Server templating tools:(Docker, Packer, Vagrant.)** Server templating tools are great for creating VMs and containers.
5. **Orchestration tools:(Kubernetes, Marathon/Mesos, Docker Swarm, Nomad)**:automate the deployment, management, scaling, and networking of containers across a cluster of machines

Introduction to Terraform

What is Terraform?

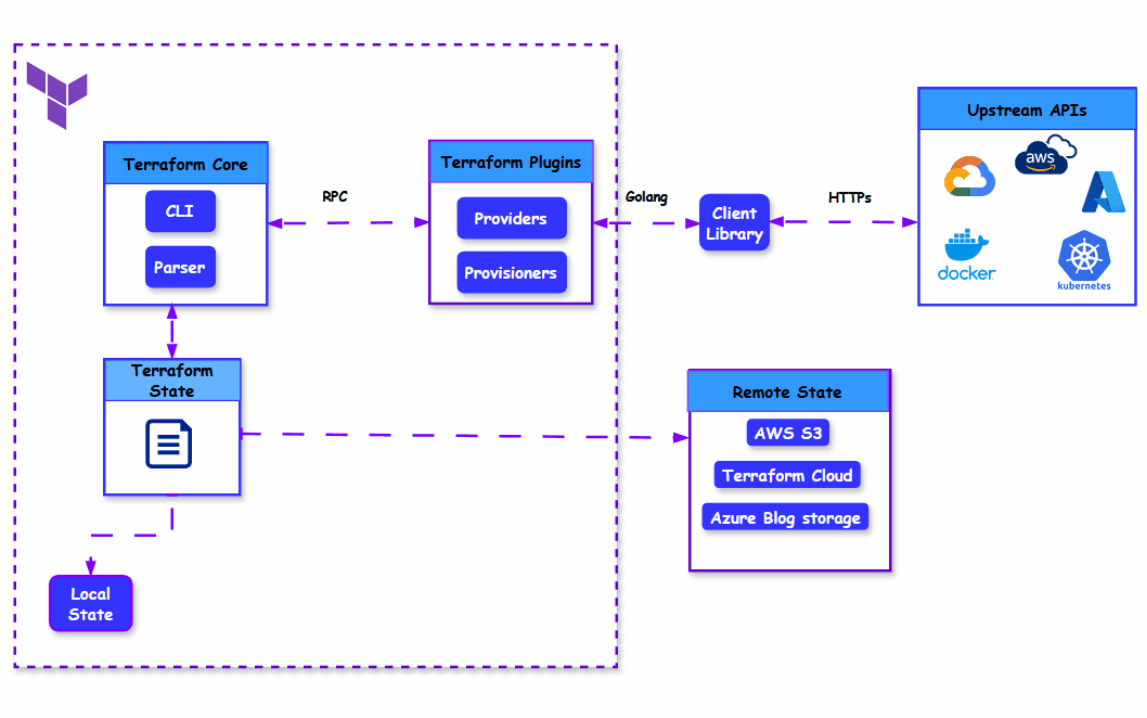
- Terraform is an infrastructure as code tool that lets you build, change, and version infrastructure safely and efficiently.
- Terraform is an open-source tool developed by HashiCorp and the most popular and widely used IaC tool.

Introduction to Terraform

Why Terraform?

- Terraform is widely used because of its **declarative syntax, platform agnostic** and its **simplicity.**
- **Multi-Cloud & Provider Support:** Terraform manages resources across many different cloud providers and services (AWS, GCP, Azure, Kubernetes, etc.) using a single, consistent workflow
- **Modular & Reusable Components:** You can create reusable modules to standardize infrastructure components across the organization.

Terraform Architecture



Terraform Architecture

- Terraform architecture mainly consists of the following components:
- Terraform core:
- Plugins (Providers and Provisioners)
- Terraform State:

Terraform Architecture

Terraform core

- Terraform core is the engine/brain behind how terraform works.
- It is responsible for reading configurations files , building the dependency graphs from resources and data sources, managing state and applying changes. Terraform Core does not directly interact with cloud providers but communicates with plugins via remote procedure calls (RPCs) and the plugins in turn communicates with their corresponding platforms via HTTPs.

Terraform Architecture

Plugins (Providers and Provisioners)

- Terraform ability is enhanced by plugins, which enable terraform to interact with cloud services and configure resources dynamically.
- Plugins acts as connectors or the glue between terraform and external APIs such as AWS, Azure, GCP, Kubernetes, Docker etc.
- Each plugin is written in the Go programming language and implements a specific interface. Terraform core knows how to install and execute plugins.
- Provisioners in Terraform are used to execute scripts or commands on a resource after it has been created or modified.

Terraform Architecture

Terraform State

- State is one of the most important core components of Terraform.
- Terraform state is a record about all the infrastructure and resources it created.
- There are two ways to manage state:
  - Local State:
  - Remote State:

Terraform Workflow

Terraform Workflow

- Terraform follows a structured execution flow to provision, update, and manage infrastructure.
  1. Write/Define Infrastructure
  2. Initialize
  3. Plan
  4. Apply
  5. Destroy

Installing Terraform

- **Installation Methods**: You can install Terraform using two methods:
  - Manual Installation using binaries
  - Using Package Managers.
- The easiest way to install Terraform is to use your operating system's package manager.
- To install terraform visit the official Hashicorp website: https://developer.hashicorp.com/terraform/install

HashiCorp Configuration Language (HCL)

- **HCL Syntax**
- **Expressions**
- **Functions**
- **Dynamic Blocks**

HCL Syntax Fundamentals

- HashiCorp Configuration Language (HCL) is Terraform's domain-specific language designed for creating structured configuration files.
- HCL is built around three key syntax constructs:
  - **Blocks:** A block acts as a containers for other content.
  - **Arguments:** Key-value pairs that assign values to specific names within a block
  - **Expressions:** Represent a value, either as a literal or a dynamic

HCL Syntax Fundamentals

HCL Syntax Rules & Conventions

- **Comments:** Supports single-line (# or //) and multi-line (/* ... */) comments.
- **Identifiers:** Names for arguments and blocks; can contain letters, digits, underscores, and hyphens. Cannot start with a digit.
- **Encoding:** Files must be **UTF-8** encoded.
- **Whitespace:** Largely ignored but used for readability. Indentation is typically two spaces.

HCL Syntax Fundamentals

HCL Expressions & Built-in Functions

- **Conditional Expressions**: Uses the ternary syntax: *condition ? true_val : false_val*
- **Interpolation**: Using *${}* to embed expressions in strings
- **Operators:** Supports arithmetic (+, -), comparison (==, !=, <, >), and logical (&&, ||, !) operations.
- **Built-in Functions:** Manipulate data using functions like upper(), lower(), lookup(), and length().
- **For Expressions:** Iterate over collections to transform data (e.g., [for s in var.list : upper(s)]

Terraform Basics

Providers

- A provider is a plugin that Terraform uses to interact with an API
- Terraform configurations must declare which providers they require so that Terraform can install and use them.
- Each provider adds a set of **resource types** and/or **data sources** that Terraform can manage.
- Providers are distributed separately from Terraform itself.
- **Terraform Registry** is the main directory of publicly available Terraform providers
- Examples of Providers: AWS, Azure, Google Cloud, Kubernetes etc.

Terraform Basics

Provider block syntax

```
provider "<PROVIDER_NAME>" {
# Configuration arguments
}
```

Example

```
provider "aws" {
  region = "us-east-1“
   -----
}
```

Terraform Basics

Resources

- A **resource** is any infrastructure object you want to create and manage with Terraform.
- Resources are the primary building blocks of infrastructure.
- Examples of resources include virtual networks, DNS, virtual servers, Database etc.
- The kinds of resources you can create depend on the providers you install.

Terraform Basics

Resource block Syntax

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {
  # Configuration arguments
}
```

Example

```
resource "aws_instance" "web" {
        ami           = "ami-123456"
        instance_type = "t2.micro"
}
```

Terraform Basics

Data Sources

- Data sources allow Terraform to query existing infrastructure without managing it.
- Data sources allow Terraform to reference external data.
- Terraform does not manage the lifecycle of the resource or data.

Terraform Basics

Data Sources Syntax

```
data "<PROVIDER>_<TYPE>" "<NAME>" {
        # Configuration arguments (filters, identifiers, etc.)
}
```

- **Example**

```
data "aws_ami" "latest" {
  most_recent = true

  owners = ["amazon"]

        filter {
        name   = "name"
        values = ["amzn2-ami-hvm-*"]
        }
}
```

Terraform Basics

Resources Vrs Data sources— when to use each

- Use resource blocks when Terraform is responsible for the entire lifecycle of an object.
- Use data source blocks to pull information from the outside world into your Terraform configuration

Terraform Basics

Variables

- Input variables are used mainly for **parameterization** making your terraform modules more flexible and reusable across different environments (dev, staging, production).
- **Variable Types**
  - Primitive Variable Types: String, Number, Boolean
  - Complex Types: List, Map, Set, Tuple, Object

Terraform Basics

Variables Syntax

```
variable "variable_name" {
        type        = string
        default     = "value"
        description = "What this variable does"
        sensitive   = false
        nullable    = true

         validation {
         condition     = length(var.variable_name) > 0
        error_message = "Value must not be empty."
        }
}
```

Example.

```
variable "instance_type" {
 type        = string
 default     = "t2.micro"
 description = "EC2 instance type"
}
```

Terraform Basics

Variables: Variable Definition Files

- Variable definition files, are used to assign actual values to the input variables you have declared in your Terraform configuration.
- Variable definition files uses the same Terraform language syntax but only consist of variable names and assignments.
- Terrafrom varible definition files names must end with either .tfvars or .tfvars.json extensions.
- Terraform automatically loads a number of variable definitions files if they are present:
  - Files named exactly terraform.tfvars or terraform.tfvars.json.
  - Any files with names ending in .auto.tfvars or .auto.tfvars.json

Terraform Basics

Variables: Variable Precedence

- When multiple varibles are provided, Terraform follows a specific precedence:
  - Environment variables (TF_VAR_name)
  - Command line flags (-var or -var-file)
  - .auto.tfvars files (alphabetical order)
  - terraform.tfvars
  - Default values in variable declarations

Terraform Basics

Outputs

- Output values provide information about your infrastructure on the command line.
- Outputs can be viewed as return values from functions in traditional programming languages.
- Outputs enable modules to share information
- Outputs serve several important purposes:
  - Providing information to the user about created resources
  - Sharing data between modules
  - Integration with other tools via output formats like JSON

Terraform Basics

Outputs Syntax

```
output "<NAME>" {
        value = <VALUE>
        [CONFIG ...]
}
```

- Example

```
output "instance_ip" {
        value = aws_instance.web.public_ip
}
```

LABs

Lab 1A. Provisioning an EC2 Instance with Terraform

- **Task.** Write Terraform configuration files to create a virtual server (EC2 instance) on AWS instead of creating resources manually through the AWS console.

- **Lab 2B. Provisioning VPC and Subnets**
- **Task:** You will write Terraform configuration files to:
  - Create a **VPC (Virtual Private Cloud)**
  - Create 1 **private subnet** within the VPC
  - Create 1 **public subnet** within the VPC